UNIVERSIDADE POSITIVO
CAMPUS PRAÇA OSÓRIO
CURSO ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

ANA JÚLIA NICOLACK
YASMIN DA ROCHA LOPES

PLAYDEX: RELATÓRIO TÉCNICO E HISTÓRICO DE DECISÕES DO PROJETO

Curitiba
2026

ANA JÚLIA NICOLACK
YASMIN DA ROCHA LOPES

PLAYDEX: RELATÓRIO TÉCNICO E HISTÓRICO DE DECISÕES DO PROJETO

Relatório técnico apresentado à disciplina de Desenvolvimento de Aplicativos Móveis do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas da Universidade Positivo, como requisito parcial à avaliação da atividade prática.

Prof. André Jeller

Curitiba

2026

RESUMO

Este relatório técnico apresenta o desenvolvimento e a evolução do **PlayDex**, uma aplicação móvel voltada para jogadores que buscam gerenciar, catalogar e compartilhar suas experiências com jogos digitais, inspirada no conceito de plataformas sociais como o *Letterboxd*. O documento detalha o histórico do projeto desde a sua concepção visual inicial (wireframes e protótipos elaborados no Trabalho 1) até a implementação funcional em código utilizando o framework **Jetpack Compose** e a linguagem **Kotlin**. São descritas a estruturação do ecossistema de telas — englobando a tela inicial, o sistema de pesquisa, os detalhes de jogos, a gestão de favoritos e o fluxo de avaliações (reviews) —, além das decisões arquiteturais tomadas pela equipe, como a centralização de rotas via NavHost e a navegação parametrizada. Por fim, o trabalho aborda os desafios técnicos enfrentados no gerenciamento de estados e listagens, evidenciando as soluções adotadas para garantir a coesão e a escalabilidade do aplicativo.

Palavras-chave: Aplicativo móvel. Jetpack Compose. Jogos digitais. Catálogo social. Análise e Desenvolvimento de Sistemas. .

SUMÁRIO

1. CONTEXTUALIZAÇÃO DO PROJETO – PLAYDEX.....14
2. HISTÓRICO E EVOLUÇÃO DO PROJETO.....16
2.1 Escopo Inicial.....16
Figura 1 — Esboços iniciais e wireframes conceituais desenhados em papel.....16
Figura 2 — Prototipagem digital das telas selecionadas no Figma (ou ferramenta similar).....17
Figura 3 — Implementação visual inicial das três telas sem interatividade.....17
2.2 Escopo Atual.....17
2.2.1 Tela Inicial.....18
Código 1 — Estruturação simplificada da Tela Inicial e suas seções de conteúdo (código resumido para fins de documentação).....19
2.2.2 Tela de Perfil.....19
Código 2 — Estruturação simplificada da Tela de Perfil e seus componentes interativos.....20
2.2.3 Tela de Registro de Review.....20
Código 3 — Estruturação simplificada da Tela de Review e seus estados dinâmicos (código resumido para fins de documentação).....22
3. IMPLEMENTAÇÃO DE NOVAS TELAS.....23
3.1. Justificativa da Escolha e Divisão de Tarefas.....23
3.2. Visão Geral das Novas Telas.....23
3.3. Aprofundamento das Novas Telas.....24
3.3.1 Tela de Pesquisa.....24
Código 4 — Estruturação simplificada da Tela de Pesquisa com gerenciamento de estado e histórico reativo (código resumido para fins de documentação).....25
Figura 4 — Interface desenvolvida para a Tela de Pesquisa.....26
3.3.2 Tela de Dados do Jogo.....26
Código 5 — Estruturação simplificada da Tela de Dados do Jogo e recuperação de informações (código resumido para fins de documentação).....27
Figura 5 — Interface detalhada e painel de informações de um jogo selecionado.28
3.3.3 Tela de Favoritos.....28
Código 6 — Estruturação simplificada da Tela de Favoritos com filtros dinâmicos (código resumido para fins de documentação).....30
Figura 6 — Interface de listagem e filtragem dos jogos favoritos.....30
3.3.4 Tela de configurações.....31
Código 7 — Estruturação simplificada da Tela de Configurações e opções de navegação (código resumido para fins de documentação).....32
Figura 7 — Interface do painel de configurações e opções do usuário.....32
3.3.5 Tela de Adicionar Favorito.....33
Código 8 — Estruturação simplificada da Tela de Adicionar Favorito e validação de dados (código resumido para fins de documentação).....34
Figura 8 — Interface e formulário implementados para a adição de novos favoritos.....35

3.3.6 Tela de Detalhes do Favorito....35
Código 9 — Estruturação simplificada da Tela de Detalhes do Favorito e exibição de parâmetros (código resumido para fins de documentação)....37
4. DECISÕES DE CONFIGURAÇÕES E ORGANIZAÇÃO DO CÓDIGO....39
4.1. Centralização e Organização das Rotas (Rotas.kt).... 39
4.2. Estruturação do Gerenciador de Rotas (NavHost e MainScreen)....39
4.3. Navegação Parametrizada e Passagem de Argumentos....39
5. COMPLEXIDADE EXTRA E NAVEGAÇÃO SECUNDÁRIA.... 41
5.1. A Escolha da Complexidade e sua Justificativa....41
6. DESAFIOS ENFRENTADOS E SOLUÇÕES ADOTADAS.... 42
6.1. O Desafio com as Listas e Navegação por Itens....42

1. CONTEXTUALIZAÇÃO DO PROJETO – PLAYDEX

O PlayDex é uma aplicação voltada para jogadores que desejam organizar, registrar e compartilhar suas experiências com jogos digitais. A proposta surgiu a partir da ideia de criar uma plataforma semelhante ao conceito de um catálogo social, como o Letterboxd, porém direcionada ao universo dos videogames.

Atualmente, jogadores costumam utilizar diferentes plataformas para acompanhar os jogos que estão jogando, registrar aqueles que já finalizaram, avaliar suas experiências e descobrir novos títulos. Essas informações podem acabar ficando distribuídas entre diferentes serviços, anotações pessoais ou até mesmo apenas na memória do próprio jogador. O PlayDex busca reunir essas funcionalidades em um único espaço, permitindo que o usuário tenha uma visão mais organizada de sua trajetória como jogador.

A aplicação foi pensada para possibilitar o registro e a organização de jogos, permitindo que o usuário mantenha sua própria coleção e acompanhe seu histórico. Além disso, o sistema também propõe recursos de avaliação, descoberta e interação, permitindo que os jogadores compartilhem suas experiências e encontrem novos jogos a partir de seus interesses.

Mais do que funcionar apenas como um catálogo, o PlayDex busca representar a relação do usuário com os jogos que fizeram parte de sua trajetória. Cada jogo registrado pode representar uma experiência, uma lembrança ou uma preferência pessoal. Dessa forma, a plataforma procura transformar uma simples lista de jogos em um espaço para construir e visualizar a própria história como jogador.

A identidade do projeto também foi desenvolvida para reforçar essa proposta. O nome PlayDex combina a ideia de *Play*, relacionada ao ato de jogar, com *Dex*, remetendo a uma coleção ou catálogo organizado. A identidade visual utiliza elementos inspirados no universo dos videogames, como controles, elementos pixelados e referências a jogos clássicos, combinados com uma estética moderna e minimalista.

Durante o desenvolvimento, o projeto passou por diferentes etapas de definição, desde a concepção da proposta e identidade visual até a elaboração das telas e funcionalidades. As decisões tomadas ao longo desse processo foram baseadas tanto na proposta inicial do

aplicativo quanto na necessidade de proporcionar uma experiência simples, organizada e visualmente coerente para o usuário.

2. HISTÓRICO E EVOLUÇÃO DO PROJETO

O desenvolvimento do *PlayDex* tem ocorrido de forma iterativa, permitindo refinar tanto a concepção conceitual quanto a implementação técnica da aplicação. Abaixo são detalhadas as principais diferenças entre a etapa inicial (Trabalho 1) e o estágio atual do projeto:

2.1 Escopo Inicial

No Trabalho 1, o escopo do projeto concentrou-se na concepção visual da interface e na estruturação inicial do layout. O processo de design seguiu etapas progressivas de prototipagem: inicialmente, foram elaborados **dez rascunhos em papel** para servir como esboço estrutural das telas.



Figura 1 — Esboços iniciais e wireframes conceituais desenhados em papel.

A partir desses esboços, cinco telas foram selecionadas e refinadas para a construção de **mockups digitais**, consolidando a identidade visual preliminar da aplicação.

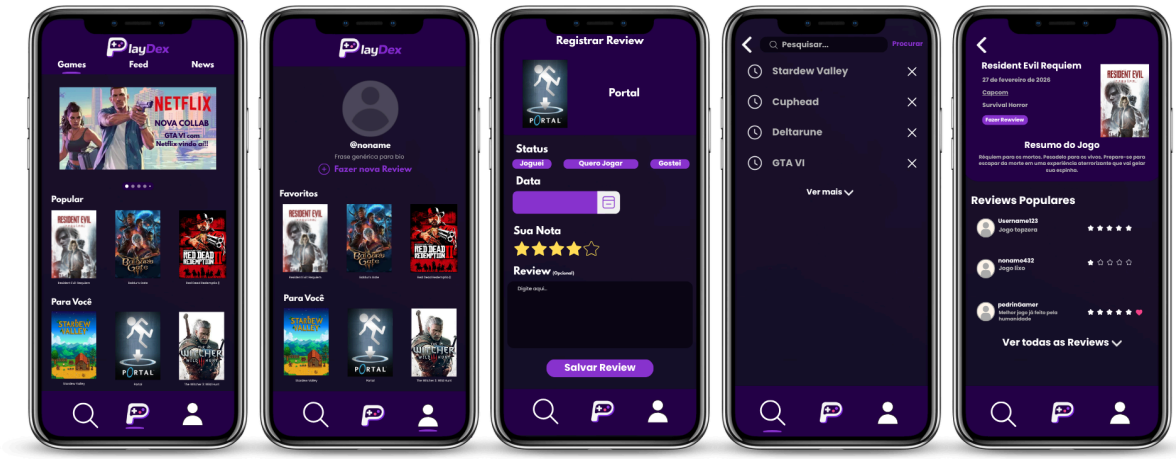


Figura 2 — Prototipagem digital das telas selecionadas no Figma (ou ferramenta similar).

Com base nos mockups, a etapa prática inicial consistiu na programação de **três telas estáticas**. O objetivo dessa entrega preliminar era focar exclusivamente na fidelidade visual e na organização dos elementos em código, de modo que os botões e componentes ainda não possuíam lógica de navegação ou interatividade dinâmica.

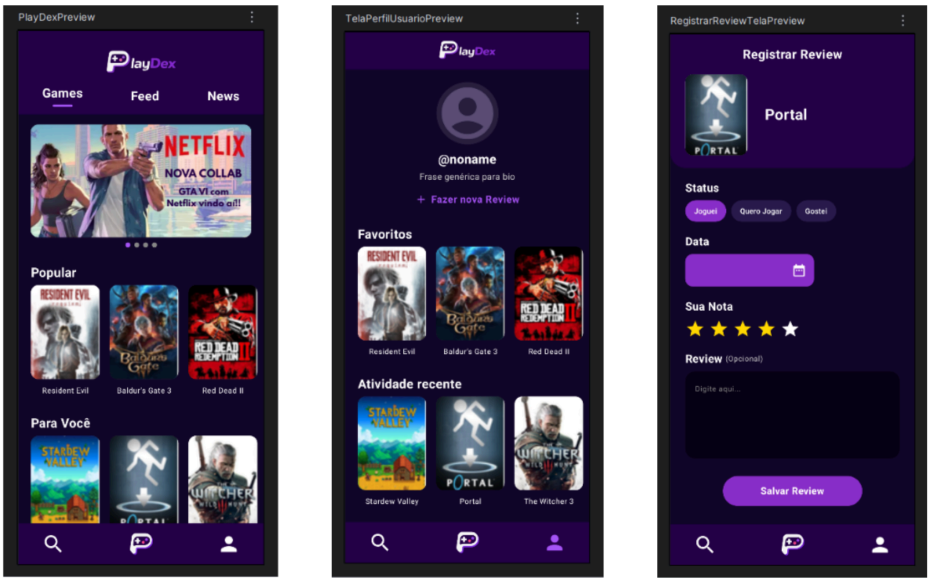


Figura 3 — Implementação visual inicial das três telas sem interatividade.

2.2 Escopo Atual

Nesta etapa atual do projeto, o foco direcionou-se para a implementação de lógica e interatividade, transformando os layouts estáticos anteriores em um fluxo navegável e coeso.

Partindo das três telas iniciais já desenvolvidas, o objetivo passou a ser conectar os componentes visuais por meio de transições e gerenciamento de estados.

2.2.1 Tela Inicial

A Tela Inicial (`TelaHome`), que anteriormente possuía apenas elementos visuais estáticos e sem comportamento funcional, foi integrada à lógica interativa do aplicativo. Por ser a interface principal exibida no carregamento do sistema, ela serve como o ponto central de navegação e consumo de conteúdo.

Nesta fase, a interface foi estruturada para organizar as seções de destaque do usuário, englobando um cabeçalho superior customizado (`TopHeaderSection`) com atalhos para configurações, uma seção de banner para notícias em destaque (`BannerSection`) e listagens dinâmicas de jogos divididas entre as categorias "Popular" e "Para Você" (`listaJogos`).

Abaixo, destaca-se a implementação estrutural da Tela Inicial e seus principais componentes visuais:

```
@Composable
fun TelaHome(onNavigateToSettings: () -> Unit) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .verticalScroll(rememberScrollState()),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        // Cabeçalho superior com logo e botão de configurações
        TopHeaderSection(onNavigateToSettings = onNavigateToSettings)

        Spacer(modifier = Modifier.height(16.dp))

        // Banner principal de novidades
        BannerSection()

        Spacer(modifier = Modifier.height(20.dp))

        // Listagem de jogos populares
        titulosJogos(title = "Popular")
        listaJogos(
            games = listOf(
                GameItem("Resident Evil", R.drawable.residentevil9),
                GameItem("Baldur's Gate 3", R.drawable.galdursgate3),
                GameItem("Red Dead II", R.drawable.reddead2)
```

```
            )
        )

        Spacer(modifier = Modifier.height(20.dp))

        // Listagem de jogos recomendados para o usuário
        titulosJogos(title = "Para Você")
        listaJogos(
            games = listOf(
                GameItem("Stardew Valley", R.drawable.stardewvalley),
                GameItem("Portal", R.drawable.portal),
                GameItem("The Witcher 3", R.drawable.thewitcher3)
            )
        )
    }
}
```

Código 1 — Estruturação simplificada da Tela Inicial e suas seções de conteúdo (código resumido para fins de documentação).

A transição fluida entre a Tela Inicial, a Tela de Pesquisa e a Tela de Perfil pode ser observada em execução na gravação de tela a seguir, evidenciando o comportamento interativo da barra de navegação: Vídeo 1 - Demonstração da Navegação.mp4

2.2.2 Tela de Perfil

A Tela de Perfil foi estruturada para centralizar as informações da conta do jogador, exibindo o avatar, nome de usuário, bio customizável e listas organizadas de jogos em destaque (divididas entre "Favoritos" e "Atividade recente").

Diferente da etapa anterior — onde a tela era puramente estática —, nesta versão atual o perfil ganhou elementos interativos funcionais, como o botão de configurações localizado no canto superior e a funcionalidade de toque nas seções (como o botão "Ver mais" em favoritos), permitindo futuras expansões de navegação.

```
@Composable
fun TelaPerfilUsuario(
    onNavigateToSettings: () -> Unit = {},
    onNavigateToFavorites: () -> Unit = {}
) {
    Column(
        modifier = Modifier.verticalScroll(rememberScrollState()),
        horizontalAlignment = Alignment.CenterHorizontally
```

```
    ) {
        // Cabeçalho com o logotipo e botão de Configurações
        Box(modifier = Modifier.fillMaxWidth()) {
            Image(painter = painterResource(R.drawable.logoplaydex),
contentDescription = "Logo")

            IconButton(onClick = onNavigateToSettings) {
                Icon(imageVector = Icons.Default.Settings, contentDescription =
"Configurações")
            }
        }

        // Seção com as informações do usuário (Avatar, Nome e Bio)
        ProfileHeaderSection()

        // Seção de Jogos Favoritos com suporte ao clique de "Ver mais"
        ProfileSectionTitle(
            title = "Favoritos",
            onSeeMoreClick = onNavigateToFavorites
        )
        ProfileGameList(games = listaDeFavoritos)

        // Seção de Atividade Recente
        ProfileSectionTitle(title = "Atividade recente")
        ProfileGameList(games = listaDeAtividades)
    }
}
```

Código 2 — Estruturação simplificada da Tela de Perfil e seus componentes interativos.

O comportamento interativo da Tela de Perfil — incluindo o acionamento do botão de configurações no cabeçalho e a navegação por meio das seções — pode ser verificado na gravação de tela apresentada a seguir: Vídeo 2 - Gravação da Tela de Perfil.mp4

2.2.3 Tela de Registro de Review

A Tela de Review foi projetada para permitir que o usuário registre interativamente suas avaliações sobre os jogos escolhidos. Enquanto na etapa anterior (Trabalho 1) a interface exibia apenas campos visuais estáticos sem comportamento funcional, nesta versão atual implementou-se a lógica completa de captura de dados, englobando botões de seleção de status, um sistema interativo de classificação por notas em formato de estrelas e a integração com um componente de calendário (*DatePicker*) para o registro da data.

Para ilustrar a gestão do estado e o salvamento interativo dos dados informados pelo usuário, destaca-se a estrutura principal da tela de avaliação:

```
@Composable
fun TelaReview(
    nomeJogo: String,
    navController: NavHostController
) {
    var selectedStatus by remember { mutableStateOf("Joguei") }
    var rating by remember { mutableStateOf(4) }
    var reviewText by remember { mutableStateOf("") }
    var selectedDate by remember { mutableStateOf("") }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .verticalScroll(rememberScrollState()),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        // Cabeçalho dinâmico contendo a capa do jogo selecionado e botão de
retorno
        HeaderReview(
            nomeJogo = nomeJogo,
            onBackClick = { navController.popBackStack() }
        )

        // Seção de Status (Joguei, Quero Jogar, Gostei)
        Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
            listOf("Joguei", "Quero Jogar", "Gostei").forEach { status ->
                Button(onClick = { selectedStatus = status }) {
                    Text(text = status)
                }
            }
        }

        // Sistema interativo de avaliação por notas (Estrelas)
        Row(horizontalArrangement = Arrangement.spacedBy(6.dp)) {
            for (i in 1..5) {
                Icon(
                    imageVector = if (i <= rating) Icons.Filled.Star else
Icons.Outlined.Star,
                    contentDescription = "Nota $i",
                    modifier = Modifier.clickable { rating = i }
                )
            }
        }

        // Botão de Confirmação e Salvamento da Review
        Button(
            onClick = {
                val novaReview = ReviewSalva(
```

```
                    nomeJogo = nomeJogo,
                    status = selectedStatus,
                    data = selectedDate,
                    nota = rating,
                    comentario = reviewText
                )
                ReviewsTemporarias.reviews.add(novaReview)
                navController.popBackStack()
            }
        ) {
            Text(text = "Salvar Review")
        }
    }
}
```

Código 3 — Estruturação simplificada da Tela de Review e seus estados dinâmicos (código resumido para fins de documentação).

O comportamento dinâmico e o fluxo de preenchimento dos campos, seleção de notas e persistência da review podem ser visualizados em execução na gravação de tela abaixo: Vídeo 3 - Gravação da Tela de Review.mp4

3. IMPLEMENTAÇÃO DE NOVAS TELAS

Nesta etapa do projeto, além de refinar as telas iniciais, o grupo expandiu o escopo do aplicativo com o desenvolvimento de seis novas interfaces fundamentais para a experiência do usuário.

3.1. Justificativa da Escolha e Divisão de Tarefas

A seleção destas cinco telas específicas — **Pesquisa**, **Dados do Jogo**, **Favoritos**, **Configurações, Adicionar Favorito** e **Detalhes Favorito** — foi motivada por critérios técnicos e estratégicos definidos pela equipe:

- **Complexidade e Requisitos Acadêmicos:** As telas foram escolhidas por atenderem perfeitamente ao nível de complexidade exigido pelo professor para a entrega atual, envolvendo lógica de filtragem, navegação parametrizada e estruturação de dados.
- **Coesão e Ciclo de Navegação:** Juntas, essas telas formam um ecossistema coeso que faz sentido dentro da jornada do usuário no catálogo social, complementando as funções de feed, perfil e review já desenvolvidas.
- **Viabilidade de Execução:** Dentre todas as ideias e rascunhos concebidos na fase inicial, este grupo de telas apresentou o melhor equilíbrio de viabilidade técnica, permitindo uma implementação sólida e bem estruturada dentro do prazo.

Para otimizar o fluxo de desenvolvimento entre os integrantes, a construção das novas telas foi dividida da seguinte forma:

- **Ana Júlia:** Desenvolvimento da Tela de Pesquisa e da Tela de Dados do Jogo.
- **Yasmin:** Desenvolvimento da Tela de Favoritos, da Tela de Configurações, da Tela de Adicionar Favoritos e Tela de Detalhes Favorito.

3.2. Visão Geral das Novas Telas

- **Tela de Pesquisa:** Desenvolvida para permitir que o usuário busque rapidamente por títulos específicos no catálogo, facilitando a descoberta de novos jogos e o acesso direto às páginas de detalhe ou avaliação.
- **Tela de Dados do Jogo:** Atua como o painel detalhado de um título selecionado, exibindo sinopse, informações de desenvolvedora, gêneros e avaliações populares associadas àquele jogo específico.

- **Tela de Favoritos:** Funciona como uma extensão organizada do perfil do usuário, listando de forma dedicada todos os jogos marcados como favoritos para consulta ágil.
- **Tela de Configurações:** Projetada para centralizar as preferências do sistema, gerenciar ajustes da conta e personalizar a experiência de uso dentro do aplicativo.
- **Tela de Adicionar Favoritos:** Responsável por adicionar itens à lista de favoritos, permitindo sua exibição dinâmica na tela correspondente.
- **Tela de Detalhes Favorito:** Responsável por persistir e gerenciar as informações dos jogos salvos na lista de favoritos.

3.3. Aprofundamento das Novas Telas

Nesta subseção, detalha-se a implementação técnica de cada uma das seis telas adicionadas ao ecossistema do *PlayDex*, evidenciando o uso de componentes dinâmicos, gerenciamento de listas e navegação por parâmetros.

3.3.1 Tela de Pesquisa

A Tela de Pesquisa foi implementada para proporcionar uma experiência dinâmica de descoberta de títulos no catálogo do aplicativo. A interface é composta por uma barra de pesquisa superior integrada a um botão de retorno e um botão de ação "Procurar", permitindo que o usuário insira termos de busca.

Além disso, o sistema gerencia um estado reativo de histórico de pesquisas (historico), onde novos termos pesquisados são adicionados ao topo da lista de forma automatizada, e os itens podem ser removidos individualmente pelo usuário.

O trecho a seguir ilustra a estruturação simplificada da lógica de pesquisa e gerenciamento do histórico:

```
@Composable
fun TelaPesquisa(
    onJogoClick: (String) -> Unit = {},
    onBackClick: () -> Unit = {}
) {
    var textoPesquisa by remember { mutableStateOf("") }
    var historico by remember { mutableStateOf(listOf("Stardew Valley", "Cuphead",
"Deltarune", "GTA VI")) }
    Column(modifier = Modifier.fillMaxSize()) {
        // Barra de Pesquisa Superior e Botão de Ação
        Row(verticalAlignment = Alignment.CenterVertically) {
```

```
            Icon(
                imageVector = Icons.Default.ArrowBack,
                contentDescription = "Voltar",
                modifier = Modifier.clickable { onBackClick() }

            BasicTextField(
                value = textoPesquisa,
                onValueChange = { textoPesquisa = it },
                decorationBox = { campoTexto ->
                    if (textoPesquisa.isEmpty()) {
                        Text(text = "Pesquisar...")
                    }
                    campoTexto()
                }
            )
            Text(
                text = "Procurar",
                modifier = Modifier.clickable {
                    val pesquisaTrim = textoPesquisa.trim()
                    if (pesquisaTrim.isNotEmpty()) {
                        historico = listOf(pesquisaTrim) + historico.filter { it !=
pesquisaTrim }
                        textoPesquisa = ""
                    }
                }
            )
        }
        // Listagem dinâmica e reativa do histórico de pesquisas
        historico.forEach { jogo ->
            ItemHistorico(
                nome = jogo,
                onClick = { onJogoClick(jogo) },
                onRemover = { historico = historico.filter { it != jogo } }
            )
        }
    }
}
```

Código 4 — Estruturação simplificada da Tela de Pesquisa com gerenciamento de estado e histórico reativo (código resumido para fins de documentação).

O layout desenvolvido para a interface de busca, contemplando a barra de pesquisa superior, o botão de ação e a listagem interativa do histórico, pode ser visualizado na figura a seguir:



Figura 4 — Interface desenvolvida para a Tela de Pesquisa.

A execução prática da busca de jogos e a manipulação interativa do histórico podem ser observadas na gravação de tela abaixo: Vídeo 4 - Gravação da Tela de Pesquisa.mp4

3.3.2 Tela de Dados do Jogo

A Tela de Dados do Jogo foi projetada para atuar como o painel detalhado de um título selecionado dentro da plataforma. Ao acessar a interface, o sistema localiza as informações correspondentes ao jogo escolhido por meio de uma estrutura de dados dedicada, exibindo o nome, a desenvolvedora, o gênero, o resumo sinóptico e a seção de avaliações populares realizadas pela comunidade.

O trecho a seguir ilustra a estruturação simplificada da recuperação dos dados e montagem do layout de detalhes:

```
@Composable
fun TelaDadosJogo(
```

```
    nomeJogo: String,
    onBackClick: () -> Unit,
    onReviewClick: () -> Unit
) {
    val jogo = buscarDadosJogo(nomeJogo)

    Column(
        modifier = Modifier
            .fillMaxSize()
            .verticalScroll(rememberScrollState())
            .padding(horizontal = 20.dp)
    ) {
        // Botão de retorno
        Icon(
            imageVector = Icons.Default.ArrowBack,
            contentDescription = "Voltar",
            modifier = Modifier.clickable { onBackClick() }
        )
        Spacer(modifier = Modifier.height(18.dp))
        // Exibição dos dados principais (Nome, Desenvolvedora, Gênero e Capa)
        DadosPrincipaisJogo(jogo = jogo)

        Spacer(modifier = Modifier.height(16.dp))
        // Botão de redirecionamento para o fluxo de review
        Button(onClick = onReviewClick) {
            Text(text = "Fazer Review")
        }

        Spacer(modifier = Modifier.height(22.dp))
        // Seção de Resumo e Sinopse
        Text(text = "Resumo do Jogo", fontSize = 21.sp, fontWeight =
FontWeight.SemiBold)
        Text(text = jogo.resumo)

        Spacer(modifier = Modifier.height(20.dp))

        // Listagem de Reviews Populares associadas
        Text(text = "Reviews Populares", fontSize = 20.sp, fontWeight =
FontWeight.SemiBold)
        ItemReview(nome = "user123", comentario = "Jogo top!", estrelas = 5)
    }
}
```

Código 5 — Estruturação simplificada da Tela de Dados do Jogo e recuperação de informações (código resumido para fins de documentação).

A concepção visual e a estruturação dos componentes gráficos da Tela de Dados do Jogo — organizados para apresentar de forma clara a sinopse e o feedback da comunidade — são apresentadas na imagem abaixo:



Figura 5 — Interface detalhada e painel de informações de um jogo selecionado.

A navegação para os detalhes de um título específico e a visualização interativa do painel podem ser conferidas na gravação de tela a seguir: Video 5 - Gravação da Tela de Dados do Jogo.mp4

3.3.3 Tela de Favoritos

A Tela de Favoritos foi projetada para funcionar como uma extensão organizada e dedicada do perfil do usuário, centralizando os títulos de maior preferência. A interface engloba um sistema de filtragem dinâmica por status ("Todos", "Joguei", "Quero jogar", "Gostei") utilizando componentes de chips interativos (`FilterChip`). Além disso, cada item da listagem exibe a capa do jogo, o status atual, trechos da avaliação cadastrada e um botão interativo para alternar o favorito em tempo real.

O trecho a seguir ilustra a estruturação simplificada da filtragem e renderização dos cards de favoritos:

```
@Composable
fun TelaFavoritos(
    onBackClick: () -> Unit = {}
) {
    val listaOriginal = remember {
        mutableStateListOf(
            FavoritoItem(1, "Resident Evil", "Joguei", "Jogo incrível com muita
tensão...", R.drawable.residentevil9),
            FavoritoItem(2, "Baldur's Gate 3", "Quero jogar", "Ansiosa para
jogar...", R.drawable.galdursgate3)
        )
    }

    var filtroSelecionado by remember { mutableStateOf("Todos") }
    val opcoesFiltro = listOf("Todos", "Joguei", "Quero jogar", "Gostei")

    val listaFiltrada = if (filtroSelecionado == "Todos") {
        listaOriginal
    } else {
        listaOriginal.filter { it.status.equals(filtroSelecionado, ignoreCase =
true) }
    }

    Column(modifier = Modifier.fillMaxSize()) {
        // Seção de Filtros com Chips dinâmicos
        LazyRow {
            items(opcoesFiltro) { filtro ->
                FilterChip(
                    selected = filtroSelecionado == filtro,
                    onClick = { filtroSelecionado = filtro },
                    label = { Text(filtro) }
                )
            }
        }

        // Listagem dos Favoritos Filtrados
        LazyColumn {
            items(listaFiltrada) { item ->
                CardFavoritoItem(
                    item = item,
                    onFavoritoToggle = {
                        val index = listaOriginal.indexOfFirst { it.id ==
item.id }
                        if (index != -1) {
                            listaOriginal[index] = listaOriginal[index].copy(
                                isFavorite = !listaOriginal[index].isFavorite
```

```
                        )
                    }
                }
            )
        }
    }
}
```

Código 6 — Estruturação simplificada da Tela de Favoritos com filtros dinâmicos (código resumido para fins de documentação).

A concepção visual e a estruturação dos componentes gráficos da Tela de Favoritos — organizados para agrupar as preferências do usuário com opções de filtragem ágil — são apresentadas na imagem abaixo:



Figura 6 — Interface de listagem e filtragem dos jogos favoritos.

O funcionamento prático da seleção de filtros e da manipulação dos cards de favoritos pode ser conferido na gravação de tela a seguir: Vídeo 6 - Gravação da Tela de Favoritos.mp4

3.3.4 Tela de configurações

A Tela de Configurações foi projetada para centralizar as preferências do sistema e os ajustes de conta do usuário dentro do aplicativo. A interface é composta por um painel de perfil simplificado no topo e uma lista de opções organizadas em cartões interativos (Surface), englobando seções como informações da conta, notificações, atalho direto para a tela de favoritos e a opção de encerramento de sessão.

O trecho a seguir ilustra a estruturação simplificada da tela de configurações e seus componentes de opções:

```
@Composable
fun TelaConfiguracoes(
    onBackClick: () -> Unit = {},
    onNavigateToFavorites: () -> Unit = {}
) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .verticalScroll(rememberScrollState()),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        // Cabeçalho com botão de retorno e título
        Box(modifier = Modifier.fillMaxWidth()) {
            IconButton(onClick = onBackClick) {
                Icon(imageVector = Icons.AutoMirrored.Filled.ArrowBack,
contentDescription = "Voltar")
            }
            Text(text = "Configurações")
        }
        Spacer(modifier = Modifier.height(24.dp))
        // Exibição do Avatar e Identificador do Usuário
        Box(modifier = Modifier.size(90.dp)) {
            Icon(imageVector = Icons.Default.AccountCircle, contentDescription =
"Avatar")
        }
        Text(text = "@username")

        Spacer(modifier = Modifier.height(30.dp))
        // Lista de Opções de Configuração
        Column(modifier = Modifier.padding(horizontal = 20.dp)) {
            ItemOpcaoConfiguracao(icon = Icons.Default.Person, titulo =
"Informações da Conta", onClick = {})
            ItemOpcaoConfiguracao(icon = Icons.Default.Notifications, titulo =
"Notificações", onClick = {})
            ItemOpcaoConfiguracao(icon = Icons.Default.Favorite, titulo =
"Favoritos", onClick = onNavigateToFavorites)
```

```
            ItemOpcaoConfiguracao(icon = Icons.AutoMirrored.Filled.ExitToApp,
titulo = "Sair", onClick = {})
        }
    }
}
```

Código 7 — Estruturação simplificada da Tela de Configurações e opções de navegação (código resumido para fins de documentação).

A concepção visual e a estruturação dos componentes gráficos da Tela de Configurações — organizados para proporcionar um gerenciamento ágil das preferências e da conta — são apresentadas na imagem abaixo:

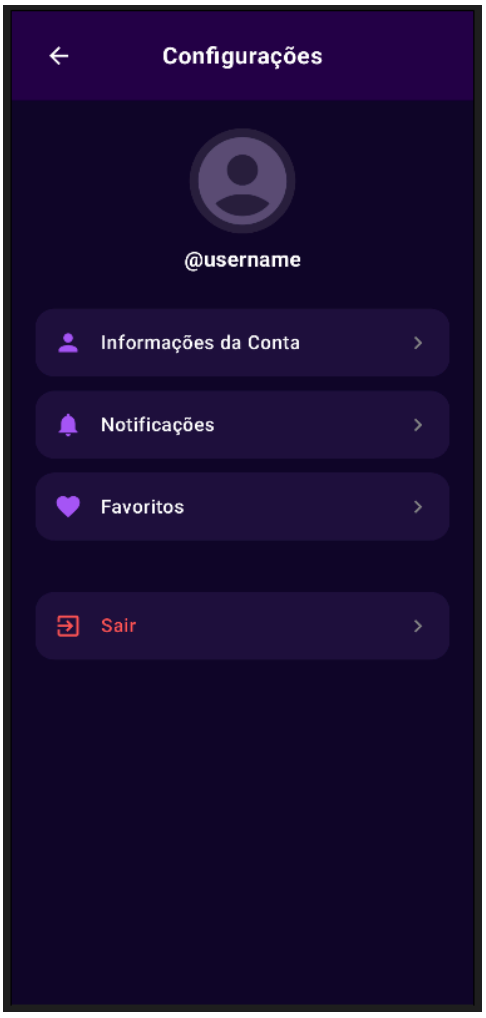


Figura 7 — Interface do painel de configurações e opções do usuário.

O funcionamento prático do acesso às opções de configuração e o acionamento dos redirecionamentos podem ser conferidos na gravação de tela a seguir: Vídeo 7 - Gravação da Tela de Configurações.mp4

3.3.5 Tela de Adicionar Favorito

A Tela de Adicionar Favorito foi projetada para expandir as funcionalidades de gerenciamento pessoal do usuário, permitindo o cadastro de novos títulos diretamente em sua coleção de preferências. A interface engloba campos de entrada para o nome do jogo, botões de seleção de status, um componente de calendário (*DatePicker*) integrado para o registro de datas, um sistema interativo de notas por estrelas e um campo dedicado a observações e *reviews*. Além disso, a tela executa validações prévias para garantir que os campos obrigatórios estejam preenchidos antes de salvar.

O trecho a seguir ilustra a estruturação simplificada da tela de cadastro e a manipulação dos estados dos campos:

```
@Composable
fun TelaAdicionarFavorito(
    onBackClick: () -> Unit,
    onAdicionar: (String, String, String, Int, String) -> Unit
) {
    var nome by remember { mutableStateOf("") }
    var status by remember { mutableStateOf("Joguei") }
    var data by remember { mutableStateOf("") }
    var nota by remember { mutableIntStateOf(0) }
    var review by remember { mutableStateOf("") }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .verticalScroll(rememberScrollState()),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        // Cabeçalho com botão de retorno e título
        Box(modifier = Modifier.fillMaxWidth()) {
            IconButton(onClick = onBackClick) {
                Icon(imageVector = Icons.AutoMirrored.Filled.ArrowBack,
contentDescription = "Voltar")
            }
            Text(text = "Adicionar Favorito")
        }

        Column(modifier = Modifier.padding(24.dp)) {
```

```
            // Campo de Entrada para o Nome do Jogo
            OutlinedTextField(
                value = nome,
                onValueChange = { nome = it },
                placeholder = { Text("Ex: Minecraft") }
            )

            // Botões de Seleção de Status
            Row {
                listOf("Joguei", "Quero jogar", "Gostei").forEach { opcao ->
                    Button(onClick = { status = opcao }) {
                        Text(opcao)
                    }
                }
            }

            // Sistema Interativo de Notas por Estrelas
            Row {
                for (estrela in 1..5) {
                    Icon(
                        imageVector = if (estrela <= nota) Icons.Filled.Star
else Icons.Outlined.Star,
                        contentDescription = "Nota $estrela",
                        modifier = Modifier.clickable { nota = estrela }
                    )
                }
            }

            // Botão de Confirmação e Inclusão do Favorito
            Button(
                onClick = {
                    if (nome.isNotBlank() && data.isNotBlank()) {
                        onAdicionar(nome.trim(), status, data, nota,
review.trim())
                    }
                }
            ) {
                Text(text = "Adicionar aos Favoritos")
            }
        }
    }
}
```

Código 8 — Estruturação simplificada da Tela de Adicionar Favorito e validação de dados (código resumido para fins de documentação).

A concepção visual e a estruturação dos componentes gráficos da Tela de Adicionar Favorito — organizados para proporcionar um cadastro intuitivo e validado — são apresentadas na imagem abaixo:

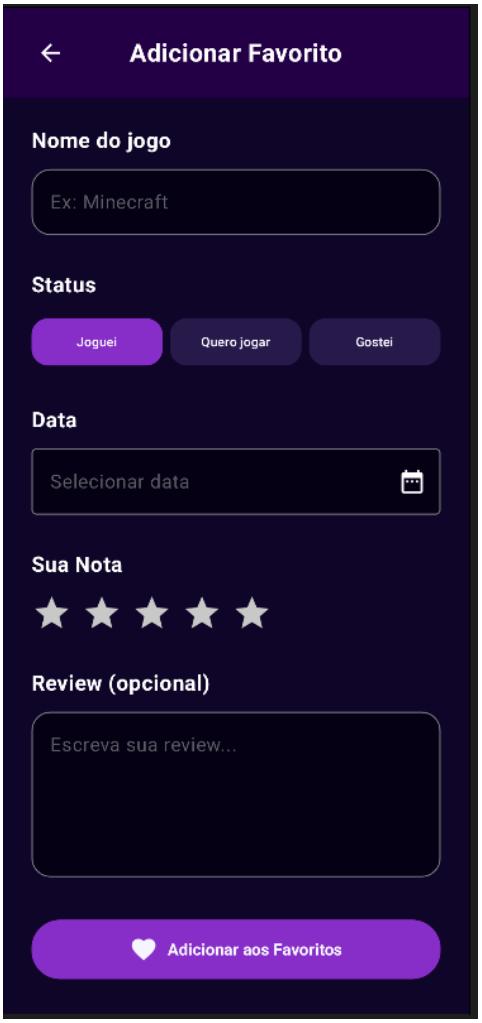


Figura 8 — Interface e formulário implementados para a adição de novos favoritos.

O funcionamento prático do preenchimento do formulário, seleção de datas e salvamento do novo jogo pode ser conferido na gravação de tela a seguir: Vídeo 8 - Gravação de Tela de Adcionar Jogo aos Favoritos.mp4

3.3.6 Tela de Detalhes do Favorito

A Tela de Detalhes do Favorito foi projetada para exibir as informações completas de um item selecionado dentro da coleção pessoal do usuário. A interface engloba um cabeçalho com botão de retorno, o título do jogo em destaque, a indicação do status atual estilizada em

cartões (Surface), a data do registro, a avaliação por notas representadas em estrelas e um bloco dedicado à exibição do texto da *review* cadastrada.

O trecho a seguir ilustra a estruturação simplificada da tela de detalhes e a renderização dos parâmetros recebidos:

```
@Composable
fun TelaDetalhesFavorito(
    nomeJogo: String,
    status: String,
    nota: Int,
    data: String,
    review: String,
    onBackClick: () -> Unit
) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .verticalScroll(rememberScrollState())
    ) {
        // Cabeçalho superior com título e botão de retorno
        Box(modifier = Modifier.fillMaxWidth()) {
            IconButton(onClick = onBackClick) {
                Icon(imageVector = Icons.AutoMirrored.Filled.ArrowBack,
contentDescription = "Voltar")
            }
            Text(text = "Detalhes do Favorito")
        }

        Column(modifier = Modifier.padding(24.dp)) {
            // Título do Jogo
            Text(text = nomeJogo, fontSize = 27.sp, fontWeight =
FontWeight.Bold)

            // Exibição do Status
            Surface(shape = RoundedCornerShape(20.dp)) {
                Text(text = status)
            }

            // Exibição da Data
            Text(text = data)

            // Exibição da Nota em Estrelas
            Text(
                text = "★".repeat(nota.coerceIn(0, 5)) + "☆".repeat(5 -
nota.coerceIn(0, 5)),
                fontSize = 28.sp
            )
```

```
            // Bloco contendo a Review Escrita
            Surface(modifier = Modifier.fillMaxWidth()) {
                Text(
                    text = review.ifBlank { "Nenhuma review foi escrita." },
                    fontSize = 14.sp
                )
            }
        }
    }
}
```

Código 9 — Estruturação simplificada da Tela de Detalhes do Favorito e exibição de parâmetros (código resumido para fins de documentação).

A concepção visual e a estruturação dos componentes gráficos da Tela de Detalhes do Favorito — organizados para apresentar de forma limpa e legível todas as informações salvas — são apresentadas na imagem abaixo:

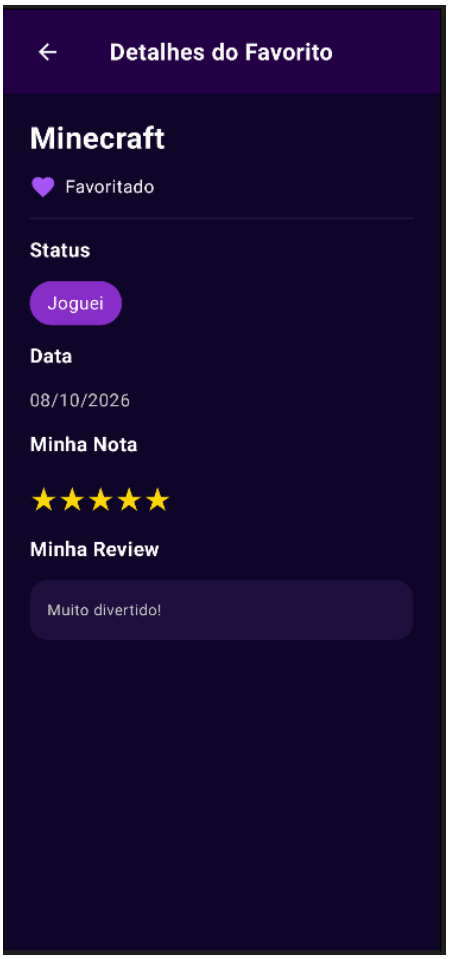


Figura 9 — Interface detalhada contendo os dados e a review do jogo favorito.

O funcionamento prático da navegação para esta tela e a visualização dos dados cadastrados podem ser conferidos na gravação de tela a seguir: Vídeo 9 - Gravação de Tela dos dados dos jogos Favoritos.mp4

4. DECISÕES DE CONFIGURAÇÕES E ORGANIZAÇÃO DO CÓDIGO

Durante o desenvolvimento do *PlayDex*, a equipe tomou decisões estratégicas de arquitetura e organização de código com o objetivo de garantir a manutenibilidade, a legibilidade e a escalabilidade da aplicação. A estrutura do projeto foi dividida em componentes modulares e focados, separando responsabilidades entre o arquivo de rotas centralizadas (Rotas.kt), a classe de controle principal (MainActivity.kt) e as telas individuais de interface.

As principais escolhas estruturais adotadas pelo trio foram:

4.1. Centralização e Organização das Rotas (Rotas.kt)

- **Decisão:** Optou-se por isolar todas as constantes de rotas do aplicativo em um objeto centralizado denominado `RotasAbas`.
- **Por quê?** Essa abordagem evita o uso de strings soltas (*magic strings*) ao longo do código de navegação. Centralizar os caminhos (como `ABA_HOME`, `TELACONFIGURACOES`, `TELA_DADOS_JOGO`, entre outros) previne erros de digitação e facilita futuras manutenções ou expansões do escopo de navegação do sistema.

4.2. Estruturação do Gerenciador de Rotas (`NavHost` e `MainScreen`)

- **Decisão:** O gerenciamento de navegação foi centralizado na função MainScreen dentro da MainActivity, unindo o componente visual Scaffold (com a barra de navegação inferior fixa `BottomNavBar`) ao componente de rotas `NavHost`.
- **Por quê?** Isolar o `NavHost` em um nível superior permite que a barra de navegação inferior persista de forma fluida entre as abas principais, enquanto gerencia o empilhamento (*back stack*) das demais telas do aplicativo de maneira controlada.

4.3. Navegação Parametrizada e Passagem de Argumentos

- **Decisão:** A navegação para telas específicas — como os detalhes de um jogo, a tela de review ou os detalhes de um favorito — foi implementada utilizando passagem de parâmetros via rota com codificação de URI (`Uri.encode`) e argumentos tipados no `NavHost`.
- **Por quê?** Isso permitiu que a aplicação fosse dinâmica, reutilizando o mesmo componente de tela (como `TelaDadosJogo` ou `TelaDetalhesFavorito`) para

exibir conteúdos diferentes com base no item selecionado pelo usuário, garantindo robustez e flexibilidade ao fluxo de dados.

5. COMPLEXIDADE EXTRA E NAVEGAÇÃO SECUNDÁRIA

Como requisito de complexidade adicional para o projeto, a equipe implementou um fluxo de navegação secundária e encadeada, conectando a **Tela de Pesquisa** aos **Detalhes do Jogo** e, por fim, à **Tela de Review**.

5.1. A Escolha da Complexidade e sua Justificativa

- **Decisão Implementada:** Adicionou-se uma rota interativa onde, ao clicar em um item pesquisado na Tela de Pesquisa, o aplicativo transita dinamicamente para a Tela de Dados do Jogo por meio de parâmetros de rota codificados. A partir dessa tela de detalhes, o usuário pode acionar o botão "Fazer Review", que o direciona para o formulário de avaliação do respectivo título.
- **Por que escolheram justamente essa?** O grupo optou por essa estrutura por ela simular o comportamento real de catálogos e plataformas sociais profissionais (como o conceito do *Letterboxd* mencionado na contextualização). Essa escolha elevou o nível técnico do trabalho ao exigir o uso de passagem de argumentos via `NavHost` (`NavType`, `Uri.encode` e captura de chaves na *back stack*), demonstrando domínio sobre o gerenciamento de estados e o fluxo de telas parametrizadas em Jetpack Compose.

6. DESAFIOS ENFRENTADOS E SOLUÇÕES ADOTADAS

Durante o processo de desenvolvimento e programação do *PlayDex*, a equipe encontrou desafios técnicos importantes, especialmente relacionados à manipulação de listas e ao fluxo de navegação detalhada. Abaixo são detalhadas as principais dificuldades e como o grupo as solucionou:

6.1. O Desafio com as Listas e Navegação por Itens

- **Dificuldade Encontrada:** Ambas as integrantes da equipe relataram desafios práticos na estruturação e no comportamento das listagens, tanto na **Tela de Pesquisa** quanto na **Tela de Favoritos**. A principal dúvida residia em como implementar a interatividade e a transição correta ao clicar em um item específico da lista, garantindo que o aplicativo respondesse exibindo os dados correspondentes.
- **Resolução na Tela de Pesquisa:** O desafio foi superado configurando o evento de clique nos itens do histórico/busca, fazendo com que o aplicativo redirecionasse o usuário de forma fluida para a página específica de dados do jogo selecionado.
- **Resolução na Tela de Favoritos:** Esse ponto exigiu uma complexidade maior, sendo resolvido com a criação de duas telas adicionais para estruturar o ciclo completo do usuário:
    1. **Tela de Adicionar Favorito:** Permite que o usuário preencha um formulário (com nome, status, data, notas e review). Ao confirmar, os dados preenchidos são salvos e a aplicação retorna para a Tela de Favoritos exibindo o novo item na lista.
    2. **Tela de Detalhes do Favorito:** Criada para solucionar a necessidade de consulta individual. Quando o usuário clica em um jogo já salvo em sua lista de favoritos, a aplicação abre esta tela dedicada, exibindo de forma detalhada todas as informações e anotações que ele havia escrito no momento do cadastro.